

Intégration Service Provider (SeP)

Solution Technique Complète

Version	1.0
Date	Mai 2026
Rôle	Service Provider (SeP) — consommateur de l'API BankID
Architecture	Gateway · Back API · Batch (Java 17 / Spring Boot 4)
Stockage données	Fichiers JSON chiffrés AES-256-GCM sur disque
Auth inter-services	JWT interne (Spring Security)

1. Contexte & Périmètre

1.1 Positionnement en tant que Service Provider

Contrairement au rôle de **banque (IDP – Identity Provider)** qui consiste à implémenter toute l'infrastructure d'authentification OpenID Connect, le rôle de **Service Provider (SeP)** est celui d'un **consommateur** de l'écosystème BankID. Le SeP utilise BankID comme source d'identité vérifiée pour ses propres clients, sans avoir à gérer la cryptographie bancaire avancée (QSEAL, etc.).

✓ Ce que vous êtes

Un Service Provider (SeP) : vous consommez l'API BankID pour vérifier l'identité de vos utilisateurs.

BankID et les banques gèrent la cryptographie lourde (JWE/JWS, QSEAL). Vous recevez des données déjà vérifiées.

Votre rôle : orchestrer le flux OAuth2/OIDC, stocker temporairement les données, les exposer en interne.

1.2 Ce que fait le système

Le flux métier complet en tant que SeP :

- L'utilisateur initie une vérification d'identité depuis votre application.
- Il est redirigé vers BankID où il s'authentifie auprès de sa banque.
- Il consent à partager les données qu'il choisit (scopes sélectionnés).
- Votre système reçoit un code d'autorisation via callback.
- Le code est échangé contre un access_token et un refresh_token.
- L'appel /userinfo ou /profile retourne les données KYC vérifiées.
- Les données sont stockées chiffrées (JSON, AES-256-GCM) sur disque pendant X jours.
- Les applications internes accèdent aux données via votre Back API (JWT interne).
- Le batch purge automatiquement les fichiers après expiration.

1.3 Stack technique

Composant	Technologie
Langage	Java 17 (LTS)
Framework	Spring Boot 4 / Spring Framework 6
Gateway	Spring Cloud Gateway
Sécurité inter-services	Spring Security 6 + JWT interne (JJWT ou Nimbus)
HTTP Client BankID	Spring WebClient (Reactor Netty, TLS)
Chiffrement fichiers	AES-256-GCM (javax.crypto)

Stockage fichiers	Système de fichiers local (JSON chiffré par sujet)
Scheduler purge	Spring @Scheduled / Spring Batch
Build	Maven 3.9+
Tests	JUnit 5, Mockito, MockWebServer

2. Démarrage & Obtention des Credentials

Contrairement au rôle de banque (enregistrement dynamique complexe avec mTLS), le SeP s'enregistre via le **portail développeur BankID**. BankID fournit directement un **client_id** et un **client_secret**.

2.1 Étapes d'enregistrement (portail developer.bankid.cz)

1. Créer un compte sur <https://developer.bankid.cz/register>
2. Créer une application (nom + logo optionnel) → accès sandbox automatique.
3. Configurer les paramètres OIDC : Redirect URIs (obligatoire), scopes requis.
4. Obtenir les credentials : client_id et client_secret (uniques par environnement).
5. Créer une organisation (nécessaire pour la production, contrats à signer).
6. Configurer l'environnement de production une fois les contrats signés.

■ Sandbox vs Production

Le client_secret sandbox et production sont DIFFÉRENTS. Ne jamais utiliser le secret sandbox en prod.

En sandbox, les utilisateurs de test sont préconfigurés par BankID.

La production nécessite une organisation créée et des contrats signés avec BankID.

2.2 Ce que vous recevez

Élément	Fourni par	Utilisation
client_id	BankID (portail)	Identifiant de votre application dans tous les appels OAuth2
client_secret	BankID (portail)	Secret partagé pour l'authentification au /token endpoint
Discovery URL	BankID (portail)	Liste toutes les URLs OIDC (auth, token, userinfo, jwks...)
Redirect URI	Vous (à configurer)	URL de callback après authentification utilisateur

2.3 Configuration Spring Boot

```
# application.yml
bankid:
  base-url: https://oidc.sandbox.bankid.cz
  client-id: ${BANKID_CLIENT_ID}
  client-secret: ${BANKID_CLIENT_SECRET}
  redirect-uri: https://votre-domaine.com/callback
  scopes: openid name birthdate addresses idcards legalstatus offline_access

kyc:
  storage-path: /data/kyc-files
  retention-days: 30

jwt:
  internal-secret: ${JWT_INTERNAL_SECRET}
  expiration-hours: 24
```

3. Architecture en 3 Projets

3.1 Vue d'ensemble

Projet	Rôle	Technologies clés
bankid-gateway	Point d'entrée unique. Route les requêtes. Valide le JWT interne sur les routes protégées. Reçoit le callback BankID et transmet au back.	Spring Cloud Gateway, Spring Security
bankid-back-api	Cœur métier. Gère le flux OAuth2 BankID, stocke les données KYC chiffrées, expose l'API interne aux autres applications.	Spring Boot, WebClient, AES-256-GCM, Spring Security
bankid-batch	Purge planifiée. Scanne les fichiers KYC, supprime ceux expirés, révoque les tokens. Journalise les opérations RGPD.	Spring Batch ou @Scheduled, WebClient (appels back API)

3.2 Flux des requêtes

Flux d'une authentification KYC complète

- ① Utilisateur → Gateway : GET /kyc/start
- ② Gateway → Back API : GET /api/kyc/auth-url (route interne)
- ③ Back API → Gateway : retourne URL /auth BankID
- ④ Gateway → Utilisateur : 302 redirect vers BankID
- ⑤ BankID → Gateway : callback GET /callback?code=...&state;=...
- ⑥ Gateway → Back API : POST /api/kyc/callback (route interne)
- ⑦ Back API → BankID : POST /token (échange code → tokens)
- ⑧ Back API → BankID : GET /userinfo (récupération données KYC)
- ⑨ Back API : chiffrement AES-256-GCM → écriture fichier JSON sur disque
- ⑩ Back API → Gateway → Utilisateur : confirmation succès

3.3 Sécurité inter-services

Le Back API n'est jamais exposé directement. Toutes les communications passent par la Gateway. Les appels du Batch vers le Back API utilisent un JWT interne signé avec un secret partagé.

```
// Génération du JWT interne (Batch → Back API)
String internalToken = Jwts.builder()
    .setSubject("bankid-batch")
    .claim("role", "INTERNAL_SERVICE")
    .setIssuedAt(new Date())
    .setExpiration(Date.from(Instant.now().plusSeconds(3600)))
    .signWith(SignatureAlgorithm.HS512, internalSecret)
    .compact();

// Validation dans Back API (Spring Security)
@PreAuthorize("hasRole('INTERNAL_SERVICE')")
public ResponseEntity<Void> triggerPurge() { ... }
```

4. Flux OAuth2/OIDC avec BankID

4.1 Configuration Spring Security OAuth2

En tant que SeP, Spring Security fournit un support natif OAuth2 Client qui simplifie grandement l'implémentation. Configuration de base :

```
# application.yml - configuration Spring OAuth2 Client
spring:
  security:
    oauth2:
      client:
        registration:
          bankid:
            client-id: ${BANKID_CLIENT_ID}
            client-secret: ${BANKID_CLIENT_SECRET}
            authorization-grant-type: authorization_code
            redirect-uri: "{baseUrl}/callback"
            scope: openid,name,birthdate,addresses,idcards,legalstatus,offline_access
        provider:
          bankid:
            issuer-uri: https://oidc.sandbox.bankid.cz
            # Spring auto-découvre les endpoints via /.well-known/openid-configuratio
n
```

4.2 Construction de l'URL d'autorisation

Le state (anti-CSRF) et le nonce sont obligatoires. Spring Security les gère automatiquement si la configuration OAuth2 Client est utilisée. En implémentation manuelle :


```
@Service
public class BankIdAuthService {

    @Value("${bankid.base-url}") private String baseUrl;
    @Value("${bankid.client-id}") private String clientId;
    @Value("${bankid.redirect-uri}") private String redirectUri;

    public String buildAuthUrl(HttpSession session) {
        String state = UUID.randomUUID().toString();
        String nonce = UUID.randomUUID().toString();
        session.setAttribute("bankid_state", state);
        session.setAttribute("bankid_nonce", nonce);

        return UriComponentsBuilder.fromHttpUrl(baseUrl + "/auth")
            .queryParams("client_id", clientId)
            .queryParams("redirect_uri", redirectUri)
            .queryParams("response_type", "code")
            .queryParams("scope", "openid name birthdate addresses idcards offline_access")
            .queryParams("state", state)
            .queryParams("nonce", nonce)
            .build().toUriString();
    }
}
```

4.3 Gestion du callback

```

@GetMapping("/callback")
public ResponseEntity<Void> handleCallback(
    @RequestParam(required = false) String code,
    @RequestParam(required = false) String state,
    @RequestParam(required = false) String error,
    @RequestParam(required = false) String error_description,
    HttpSession session) {

    // 1. Erreur BankID en priorité
    if (error != null) {
        log.warn("BankID error: {} - {}", error, error_description);
        return ResponseEntity.status(302)
            .header("Location", "/kyc/error?code=" + error).build();
    }

    // 2. Vérification anti-CSRF
    String expectedState = (String) session.getAttribute("bankid_state");
    if (!state.equals(expectedState))
        throw new SecurityException("State mismatch - possible CSRF");

    // 3. Orchestration KYC
    String subject = kycOrchestrator.processCode(code, session);
    return ResponseEntity.status(302)
        .header("Location", "/kyc/success?sub=" + subject).build();
}

```

4.4 Échange du code (Token Endpoint)

Le SeP utilise **client_secret_basic** ou **client_secret_post** (beaucoup plus simple que le **private_key_jwt** bancaire). Spring Security OAuth2 Client gère cet échange automatiquement.

```

public TokenResponse exchangeCode(String code) {
    MultiValueMap<String, String> body = new LinkedMultiValueMap<>();
    body.add("grant_type", "authorization_code");
    body.add("code", code);
    body.add("redirect_uri", redirectUri);
    body.add("client_id", clientId);
    body.add("client_secret", clientSecret);

    return webClient.post()
        .uri("/token")
        .contentType(MediaType.APPLICATION_FORM_URLENCODED)
        .bodyValue(body)
        .retrieve()
        .bodyToMono(TokenResponse.class)
        .block();
}

```

4.5 Récupération des données KYC (/userinfo)

```
public Map<String, Object> fetchUserInfo(String accessToken) {  
    return webClient.get()  
        .uri("/userinfo")  
        .header(HttpHeaders.AUTHORIZATION, "Bearer " + accessToken)  
        .retrieve()  
        .bodyToMono(new ParameterizedTypeReference<Map<String, Object>>() {})  
        .block();  
}  
  
// Note : /profile retourne un JWT signé (JWS) si configuré ainsi.  
// /userinfo retourne du JSON standard – plus simple pour un SeP.
```

5. Scopes & Données Disponibles

Les scopes définissent les données que l'utilisateur est invité à partager. Ils sont configurés dans le portail développeur et peuvent être marqués **required** (l'utilisateur ne peut pas refuser) ou **optional** (l'utilisateur peut refuser).

Scope	Claims principaux	Disponibilité banques
openid	sub (identifiant unique pairwise)	Toutes – OBLIGATOIRE
name	given_name, family_name, middle_name	Toutes les banques
birthdate	birthdate, age, date_of_death	Toutes les banques
addresses	type, street, city, zipcode, country	Toutes (détail variable)
idcards	type, number, valid_to, country, issuer	Toutes les banques
legalstatus	majority, pep, limited_legal_capacity	Toutes les banques
gender	gender	Toutes les banques
email	email, email_verified	Variable selon banque
phonenummer	phone_number, phone_number_verified	Variable selon banque
birthnumber	birthnumber (n° naissance tchèque)	Variable selon banque
birthplaceNationality	birthplace, birthcountry, nationalities	Toutes les banques
verification	trust_framework, verification_process, time	Toutes les banques
paymentAccounts	paymentAccounts, paymentAccountsDetails	AB, mB uniquement
offline_access	Activation du refresh_token	Toutes – si demandé

■ Note sur la disponibilité des claims

Y = toujours envoyé par la banque, O = envoyé si la banque possède la valeur, N = non envoyé.

Le scope 'idcards' peut être vide si l'utilisateur n'a pas de carte valide (expirée, volée...).

paymentAccounts n'est disponible que chez AB (Air Bank) et mB (mBank).

Toujours traiter les claims comme optionnels côté code – même un scope 'required' peut retourner null.

6. Stockage Chiffré des Données KYC

6.1 Structure de stockage

Chaque réponse KYC est stockée dans un fichier JSON chiffré AES-256-GCM. Le nom de fichier est basé sur le **sub** (identifiant pairwise BankID), jamais sur des données personnelles.

```
/data/kyc-files/
■■■■ {sub-hash}.kyc.enc      ← Données KYC chiffrées (JSON → AES-256-GCM → Base64)
■■■■ {sub-hash}.meta.json   ← Métadonnées non-chiffrées (dates, statut, sub hashé)
■■■■ audit/
    ■■■■ purge-audit.log    ← Journal des purges (15 ans de rétention)
```

6.2 Format du fichier .kyc.enc

Le fichier chiffré contient le JSON des claims, précédé de l'IV (12 bytes) en Base64 :

```
// Structure binaire : [IV 12 bytes][Ciphertext + GCM Tag 16 bytes]
// Encodé en Base64 pour le fichier texte

// JSON source (avant chiffrement) :
{
  "sub": "pairwise-sub-from-bankid",
  "fetchAt": "2026-05-20T10:30:00Z",
  "expiresAt": "2026-06-19T10:30:00Z",
  "bankName": "CSOB",
  "scopes": ["name", "birthdate", "addresses"],
  "claims": {
    "given_name": "Jan",
    "family_name": "Novak",
    "birthdate": "1985-03-15",
    "addresses": [{"type": "PERMANENT", "city": "Praha", ...}]
  }
}
```

6.3 Service de chiffrement AES-256-GCM

```

@Service
public class KycEncryptionService {

    @Value("${kyc.encryption.key}")
    private String base64Key; // Clé 256 bits encodée Base64 – depuis vault/env

    public void encryptAndWrite(String subject, KycData data, Path storageDir) {
        String json = objectMapper.writeValueAsString(data);
        byte[] plaintext = json.getBytes(StandardCharsets.UTF_8);

        SecretKey key = new SecretKeySpec(Base64.getDecoder().decode(base64Key), "AES");

        byte[] iv = new byte[12];
        new SecureRandom().nextBytes(iv);

        Cipher cipher = Cipher.getInstance("AES/GCM/NoPadding");
        cipher.init(Cipher.ENCRYPT_MODE, key, new GCMParameterSpec(128, iv));
        byte[] ciphertext = cipher.doFinal(plaintext);

        // IV + ciphertext concaténés, encodés Base64
        byte[] result = new byte[iv.length + ciphertext.length];
        System.arraycopy(iv, 0, result, 0, iv.length);
        System.arraycopy(ciphertext, 0, result, iv.length, ciphertext.length);

        String filename = hashSubject(subject) + ".kyc.enc";
        Files.writeString(storageDir.resolve(filename),
            Base64.getEncoder().encodeToString(result));
    }

    private String hashSubject(String sub) {
        // SHA-256 du sub pour le nom de fichier (pas de PII en clair)
        return DigestUtils.sha256Hex(sub);
    }
}

```

6.4 Fichier .meta.json (métadonnées non chiffrées)

```

{
  "subHash": "a1b2c3d4e5...", // SHA-256 du sub
  "fetchAt": "2026-05-20T10:30:00Z",
  "expiresAt": "2026-06-19T10:30:00Z",
  "status": "ACTIVE", // ACTIVE | REVOKED | EXPIRED
  "scopes": ["name", "birthdate"],
  "bankName": "CSOB"
}

```

7. API REST Interne – Exposition des Données

7.1 Endpoints disponibles

Endpoint	Méthode	Description	Auth
/api/kyc/{sub}/profile	GET	Profil KYC complet déchiffré	JWT interne
/api/kyc/{sub}/identity	GET	Nom, prénom, date de naissance	JWT interne
/api/kyc/{sub}/addresses	GET	Adresses du sujet	JWT interne
/api/kyc/{sub}/documents	GET	Documents d'identité	JWT interne
/api/kyc/{sub}/status	GET	Statut et métadonnées KYC	JWT interne
/api/kyc/{sub}	DELETE	Suppression immédiate (droit à l'oubli)	JWT interne
/api/kyc/auth-url	GET	Génère l'URL d'authentification BankID	JWT interne
/api/kyc/callback	POST	Traite le callback BankID (depuis gateway)	JWT interne
/api/kyc/purge	POST	Déclenche une purge manuelle (depuis batch)	JWT interne

7.2 DTO de réponse

```
// KycProfileDto.java
public record KycProfileDto(
    String subHash,
    IdentityDto identity,
    List<AddressDto> addresses,
    List<IdCardDto> idCards,
    ContactDto contact,
    VerificationDto verification,
    Instant fetchedAt,
    Instant expiresAt,
    String status
) {}

public record IdentityDto(
    String givenName, String familyName,
    String birthdate, Integer age,
    Boolean majority, Boolean pep
) {}

public record VerificationDto(
    String trustFramework,
    String verificationProcess,
    Instant verifiedAt
) {}
```

7.3 Controller REST (Back API)

```
@RestController
@RequestMapping("/api/kyc")
@PreAuthorize("hasRole('INTERNAL_SERVICE')")
public class KycController {

    @GetMapping("/{subHash}/profile")
    public ResponseEntity<KycProfileDto> getProfile(@PathVariable String subHash) {
        return kycStorageService.read(subHash)
            .map(data -> ResponseEntity.ok(mapper.toDto(data)))
            .orElse(ResponseEntity.notFound().build());
    }

    @DeleteMapping("/{subHash}")
    public ResponseEntity<Void> deleteKyc(
        @PathVariable String subHash,
        @RequestHeader("X-Deletion-Reason") String reason) {
        kycStorageService.delete(subHash, reason); // Log audit + suppression fichier
        return ResponseEntity.noContent().build();
    }
}
```


8. Batch – Purge Automatique RGPD

8.1 Politique de rétention

Type de données	Durée de rétention	Action à expiration
Fichiers KYC chiffrés (.kyc.enc)	X jours (configurable, défaut 30j)	Suppression physique du fichier
Fichiers de métadonnées (.meta.json)	Même durée que le fichier KYC	Suppression physique
Tokens (access / refresh)	Durée de validité + 24h	Révocation BankID + suppression locale
Logs d'audit (purge-audit.log)	15 ans minimum (obligation légale)	Archivage froid
Sessions nonce/state	30 minutes max	Invalidation automatique session

8.2 Scheduler de purge

```

@Component
public class KycPurgeScheduler {

    @Value("${kyc.storage-path}") private Path storageDir;
    @Value("${kyc.retention-days}") private int retentionDays;

    @Scheduled(cron = "0 0 2 * * *") // Chaque nuit à 2h
    public void purgeExpiredFiles() {
        LocalDateTime cutoff = LocalDateTime.now().minusDays(retentionDays);

        try (Stream<Path> files = Files.list(storageDir)) {
            files.filter(f -> f.toString().endsWith(".meta.json"))
                .forEach(metaFile -> processMetaFile(metaFile, cutoff));
        }

    }

    private void processMetaFile(Path metaFile, LocalDateTime cutoff) {
        KycMeta meta = objectMapper.readValue(metaFile.toFile(), KycMeta.class);

        if (meta.expiresAt().isBefore(cutoff)) {
            // 1. Révoquer le refresh_token si encore valide
            if (meta.refreshToken() != null) {
                bankIdClient.revokeToken(meta.refreshToken(), "refresh_token");
            }
            // 2. Supprimer les fichiers
            Files.deleteIfExists(metaFile.resolveSibling(meta.subHash() + ".kyc.enc"));
        };

        Files.deleteIfExists(metaFile);
        // 3. Journaliser (audit 15 ans)
        auditLogger.logPurge(meta.subHash(), Instant.now(), "SCHEDULED_EXPIRY");
        log.info("Purged KYC for subHash: {}", meta.subHash());
    }
}

```

8.3 Appel depuis le batch vers le Back API

Le batch ne touche jamais directement les fichiers — il appelle le Back API via REST pour conserver la séparation des responsabilités :

```
@Component
public class BatchPurgeClient {

    public void triggerPurge() {
        String internalJwt = jwtService.generateInternalToken("bankid-batch");

        webClient.post()
            .uri(backApiUrl + "/api/kyc/purge")
            .header(HttpHeaders.AUTHORIZATION, "Bearer " + internalJwt)
            .retrieve()
            .toBodilessEntity()
            .block();
    }
}
```

9. Gestion des Erreurs BankID

9.1 Codes d'erreur OAuth2 (callback)

Code erreur	Cause	Action recommandée
access_denied	L'utilisateur a refusé l'authentification ou le consentement	Rediriger vers page 'authentification refusée'
invalid_scope	Scope non supporté ou 'openid' manquant	Vérifier la configuration des scopes
eid_doesnt_exist	L'utilisateur n'a pas de compte BankID éligible	Proposer une alternative d'identification
temporarily_unavailable	BankID temporairement indisponible	Retry avec backoff exponentiel
server_error	Erreur interne BankID	Log + notification équipe + retry
invalid_request	Paramètre manquant ou invalide (nonce, state...)	Vérifier la construction de l'URL /auth
interaction_required	prompt=none impossible, interaction requise	Ne pas utiliser prompt=none pour KYC

9.2 Handler global d'erreurs

```

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(BankIdException.class)
    public ResponseEntity<ErrorResponse> handleBankIdError(BankIdException e) {
        log.warn("BankID error: {}", e.getErrorCode(), e);
        return ResponseEntity.status(switch (e.getErrorCode()) {
            case "access_denied" -> 403;
            case "temporarily_unavailable" -> 503;
            default -> 400;
        }).body(new ErrorResponse(e.getErrorCode(), e.getMessage()));
    }

    @ExceptionHandler(SecurityException.class)
    public ResponseEntity<ErrorResponse> handleSecurity(SecurityException e) {
        log.error("Security violation: {}", e.getMessage());
        return ResponseEntity.status(403)
            .body(new ErrorResponse("SECURITY_VIOLATION", "Access denied"));
    }
}

```

10. Conformité RGPD

10.1 Droits des personnes

Droit (RGPD)	Article	Implémentation
Droit d'accès	Art. 15	GET /api/kyc/{subHash}/profile → retourne les données déchiffrées
Droit à l'effacement	Art. 17	DELETE /api/kyc/{subHash} → suppression immédiate fichiers + log audit
Droit à la portabilité	Art. 20	GET /api/kyc/{subHash}/profile → format JSON standard
Droit de limitation	Art. 18	PATCH status=SUSPENDED sur le fichier meta → gel des données
Minimisation	Art. 5	Ne demander que les scopes nécessaires au traitement métier

10.2 Journalisation d'audit

Les éléments suivants DOIVENT être journalisés et conservés 15 ans :

- Date et heure de chaque récupération de données KYC.
- Identifiant de transaction BankID (claim txn) si disponible.
- Résultat de chaque opération (succès / échec / refus utilisateur).
- Toute purge ou suppression avec sa raison (SCHEDULED_EXPIRY, USER_REQUEST, ADMIN...).
- Accès aux données par les applications internes (qui, quand, quel subHash).

X Règle absolue sur les logs d'audit

Les logs d'audit NE DOIVENT JAMAIS contenir de données personnelles en clair.

Utiliser uniquement le subHash (SHA-256 du sub BankID) comme identifiant dans les logs.

Ne jamais logger nom, prénom, date de naissance, numéro de document dans les fichiers d'audit.

11. Structure des Projets Maven

11.1 Arborescence multi-modules

```

bankid-kyc-platform/                                ← Parent POM
■■■■ pom.xml
■■■■ bankid-gateway/                                ← Projet 1 : Gateway
■   ■■■■ src/main/java/.../gateway/
■   ■   ■■■■ GatewayApplication.java
■   ■   ■■■■ config/ (RouteConfig, SecurityConfig)
■   ■   ■■■■ filter/ (JwtAuthFilter, LoggingFilter)
■   ■■■■ src/main/resources/application.yml
■
■■■■ bankid-back-api/                                ← Projet 2 : Back API
■   ■■■■ src/main/java/.../backapi/
■   ■   ■■■■ BackApiApplication.java
■   ■   ■■■■ auth/ (BankIdAuthService, CallbackController)
■   ■   ■■■■ kyc/ (KycOrchestrator, KycStorageService)
■   ■   ■■■■ crypto/ (KycEncryptionService)
■   ■   ■■■■ api/ (KycController, DTOs)
■   ■   ■■■■ security/(JwtConfig, SecurityConfig)
■   ■■■■ src/main/resources/
■
■■■■ bankid-batch/                                    ← Projet 3 : Batch
■■■■ src/main/java/.../batch/
■   ■■■■ BatchApplication.java
■   ■■■■ purge/ (KycPurgeScheduler, BatchPurgeClient)
■   ■■■■ audit/ (AuditLogger)
■■■■ src/main/resources/

```

11.2 Dépendances Maven clés

Dépendance	Module	Usage
spring-boot-starter-webflux	gateway, back-api, batch	WebClient, Spring Cloud Gateway
spring-cloud-starter-gateway	gateway	Routing, filtres
spring-boot-starter-security	gateway, back-api	Spring Security OAuth2
spring-security-oauth2-client	back-api	Flux OAuth2/OIDC natif
spring-boot-starter-batch	batch	Spring Batch (jobs de purge)
jjwt-api / jjwt-impl	back-api, batch	Génération/validation JWT interne
spring-boot-starter-actuator	tous	Health checks, métriques
commons-codec	back-api	DigestUtils (SHA-256 du sub)

12. Checklist Mise en Production

Sandbox (développement)

1. Créer un compte sur `developer.bankid.cz` et une application sandbox.
2. Récupérer `client_id` et `client_secret` sandbox.
3. Configurer la Redirect URI dans le portail (ex: `http://localhost:8080/callback`).
4. Valider le flux complet : `/auth` → `callback` → `/token` → `/userinfo`.
5. Tester la gestion des erreurs : accès refusé, scope invalide.
6. Valider le chiffrement/déchiffrement des fichiers KYC.
7. Tester la purge automatique avec une rétention réduite (ex: 1 min).

Production

1. Créer une organisation sur le portail et signer les contrats BankID.
2. Configurer l'application pour l'environnement production.
3. Stocker `client_id` et `client_secret` production dans un vault (jamais en clair).
4. Configurer la Redirect URI de production (HTTPS obligatoire).
5. Générer et stocker la clé AES-256 de chiffrement dans le vault.
6. Générer et stocker le secret JWT interne dans le vault.
7. Activer HTTPS avec un certificat valide sur la Gateway.
8. Configurer la politique de rétention avec le DPO (durée X jours).
9. Mettre en place les alertes sur les erreurs BankID (`temporarily_unavailable`, `server_error`).
10. Tester le droit à l'effacement (DELETE) de bout en bout.
11. Vérifier que les logs d'audit ne contiennent aucune PII en clair.

✓ Avantage clé du rôle SeP vs Banque

Pas de certificats QWAC/QSEAL à obtenir.

Pas d'enregistrement dynamique complexe (mTLS + `private_key_jwt`).

Pas de JWE/JWS à implémenter côté serveur.

Un simple `client_id` + `client_secret` suffisent pour démarrer en sandbox.

Spring Security OAuth2 Client gère la majorité du flux automatiquement.

Annexe A – URLs de Référence

Ressource	URL
Portail développeur	https://developer.bankid.cz
Sandbox base URL	https://oidc.sandbox.bankid.cz
OpenID Discovery	https://oidc.sandbox.bankid.cz/.well-known/openid-configuration
JWK Keys	https://oidc.sandbox.bankid.cz/.well-known/jwks
Authorization	https://oidc.sandbox.bankid.cz/auth
Token	https://oidc.sandbox.bankid.cz/token
UserInfo	https://oidc.sandbox.bankid.cz/userinfo
Profile (KYC)	https://oidc.sandbox.bankid.cz/profile
Revoke	https://oidc.sandbox.bankid.cz/revoke
Doc SeP overview	https://developer.bankid.cz/docs/high_level_overview_sep
Doc SeP APIs	https://developer.bankid.cz/docs/apis_sep
Doc SeP démarrage	https://developer.bankid.cz/docs/sep_get_started
GitHub exemples Java	https://github.com/BankovníIdentita/BankID-examples/tree/master/java

Annexe B – Variables d'Environnement

Variable	Description	Sensibilité
BANKID_CLIENT_ID	Identifiant de l'application BankID	Faible (pas secret)
BANKID_CLIENT_SECRET	Secret partagé BankID	CRITIQUE – vault obligatoire
KYC_ENCRYPTION_KEY	Clé AES-256 base64 pour chiffrement fichiers	CRITIQUE – vault obligatoire
JWT_INTERNAL_SECRET	Secret HMAC pour JWT inter-services	CRITIQUE – vault obligatoire
KYC_STORAGE_PATH	Chemin disque pour les fichiers KYC	Faible
KYC_RETENTION_DAYS	Durée de rétention en jours (défaut: 30)	Faible
BACK_API_URL	URL interne du Back API (depuis batch/gateway)	Faible